

AI for Economic Research: Dynamic Models, Language, and Agents

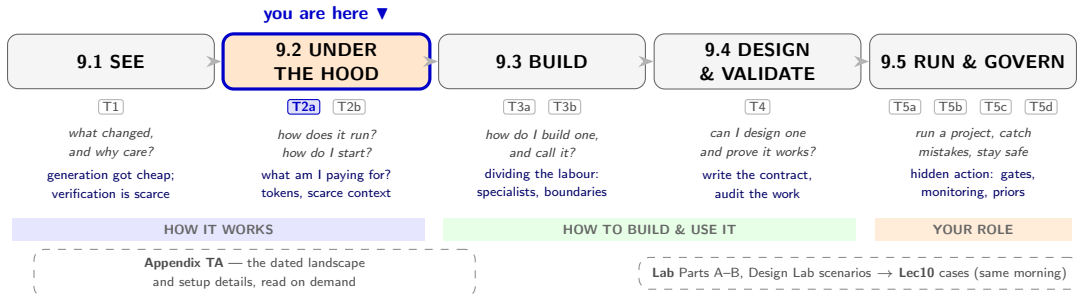
Lecture 9.2a: How an Agent Actually Runs — Harness, Session, Loop

Zhigang Feng

2026-09-12 15:59:05

Lecture 9 in One Map: How an Agent Runs

This deck answers the second question: how does it actually work, and how do I start?



Route: the three physical components and the tool-call loop → install and initialize → the shape of the CLI → your first session (on the fellows roster) → tokens, the context window, and the pillars that keep you honest.

Topic 9.2b Agenda

T2 opened the hood on one agent. This interlude answers the six questions students actually ask once they have run a session — and corrects three beliefs that are widespread and wrong.

1. **One agent, written out** — what a “session” physically is, and the loop in twenty lines of Python
2. **Three backgrounds, three agents** — the eight steps that turn a job description into a file you can dispatch
3. **Many agents** — how you create a *set* of them, and what actually limits the number
4. **Who decides to dispatch?** — the router myth, and what really happens
5. **Agent vs. skill** — a distinction most write-ups get backwards
6. **Synthesis** — and a lab where you design and dispatch your own agents

Why a separate deck. T1b surveyed the *landscape* of multi-agent systems; T4 taught you to *use* sub-agents. Neither showed the machinery, and neither had you *build* one. Everything here is checkable on your own machine — and the lab makes you check it.

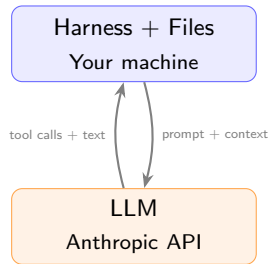
Three Components and One JSON Line

Harness, model, files — and the loop that joins them

Three Components: Harness, LLM, Your Files

A terminal agent has three physically distinct parts — the same anatomy in Claude Code, Codex CLI, Gemini CLI, or Aider:

1. **The Harness** (lives on your machine)
CLI software you install. Reads your files, runs shell commands, manages the conversation loop.
2. **The LLM** (lives on the provider's servers)
Every message is bundled with conversation history and sent via HTTPS to the API. The “thinking” happens in the cloud.
3. **Your Files** (live on your machine)
Python scripts, data CSVs, LaTeX—all on your disk. Code executes on your hardware with your packages.



Key: code can run locally and many project files stay on your machine. That does *not* mean the whole workflow is automatically private.

The Loop in Machine Language: JSON Tool Calls

Lec07 T4 explained that an LLM can emit structured JSON via schema-constrained decoding. In a terminal agent, every tool call uses that mechanism.

Example. When the agent decides to read a file, it emits:

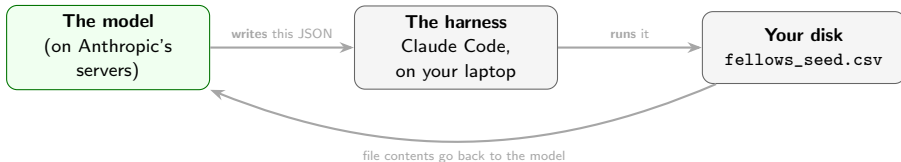
```
{"tool": "Read", "args": {"file_path": "fellows_seed.csv"}}
```

The harness parses the JSON, executes Read, and feeds the file content back as the next message; the agent reads it and decides the next action.

- **Loop.** Model output (JSON) → harness execution → tool result → next model turn. Continues until the model emits a “done” signal or hits a stop condition.
- **Schema is the contract.** Each tool advertises an argument schema; outputs that fail the schema are rejected.
- **Same brain.** There is no separate “planner” — the same transformer that writes code also decides to call tools.
- **Key implication.** Every action the agent takes is a *next-token prediction* constrained by the available tool schema.

From One JSON Line to a Running Loop

When an agent wants to read a file, it does not read the file. It writes a request — one line of JSON — and something else does the reading.



What the model emits (the line above):

```
{"tool": "Read",  
  "args": {"file_path": "..."}}
```

It is *text the model generated*, in a shape you told it to use. The model cannot touch your disk; the harness can.

The naive mental model. “I open a session, the model remembers me, and somewhere a dispatcher assigns work to specialists.”

Every clause is wrong. This deck replaces it — and the cost model, the context limits, and the sub-agent behaviour of T3b and T5b–T5c all follow.

Answered next: if nothing is stored, what is a “session” — and where is “the agent”?

Privacy Boundary: Local Execution Is Not the Same as Local Processing

- code may execute locally on your machine
- many project files may stay on disk locally
- if file contents, prompts, or tool outputs are sent back in the conversation, those contents may still traverse the network
- only a fully local or institutionally private deployment changes that boundary

Practical rule: running code locally does not make it safe to put restricted data into the conversation — treat everything you type or attach as leaving your machine.

The Session Is a List

What a session is, and the loop that drives it

First, the Vocabulary: Who Is Speaking?

Every message carries a `role` label saying who produced it. There are three things you will see, and one is misleadingly named.

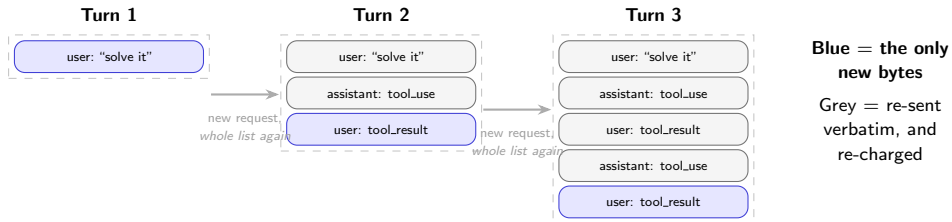
Label	Who actually wrote it	Example
"user"	you	"Solve the model and report r ."
"assistant"	the LLM — Claude itself	the JSON tool request on the last slide
"user", containing a <code>tool_result</code>	your harness, reporting what a tool returned	the contents of <code>fellows_seed.csv</code>

The naming trap. "Assistant" is simply the API's word for *the model's own turn* — it is not a separate helper program. And tool results are labelled "user" even though you did not type them: the API has only two speakers, so anything sent to the model is labelled "user", whoever produced it. Read "user" as "input to the model", not as "the human".

Why it matters: assistant means *the model*; tool output re-enters as user. That is all the vocabulary this deck needs.

The Session Is a List, Not a Connection

The API is stateless. Each turn is a separate request that carries the entire conversation so far — the model remembers nothing between calls.



- **"Re-send everything" is literal:** turn 3 is a fresh request carrying turns 1–3 in full
- No open connection, no server-side memory of you
- "Context window full" means *this list* outgrew the limit
- You are billed for the whole list each turn

Where the list starts and stops. /clear does exactly one thing: it throws the list away and starts an empty one. Compaction replaces the list with a summary of itself. Both are list operations — there is no "connection" to reset.

Four Moving Parts, One Function Call

An agent is not a service you connect to. It is four arguments you assemble and send.

Argument	What it is	Concretely
model	which LLM to send to	the string "claude-opus-5" — just a name; you are picking a brain
system	the standing instructions : role, rules, style, read <i>before</i> the conversation	"You are a computational macroeconomist. Never invent data." This is what CLAUDE.md becomes (T2b, T5b)
tools	the menu of actions — name, description, argument schema. <i>Descriptions only; no code is sent</i>	"Read: read a file. Arguments: file_path (string)." The model picks from this menu
messages	the conversation so far	every user / assistant / tool_result turn, in order

What comes back: ordinary text, or tool_use blocks — the model asking you to run something from the menu.

Change system and tools and you have a *different agent*; there is no "agent object" to create or shut down.

The Loop in Twenty Lines — Written by You, Once

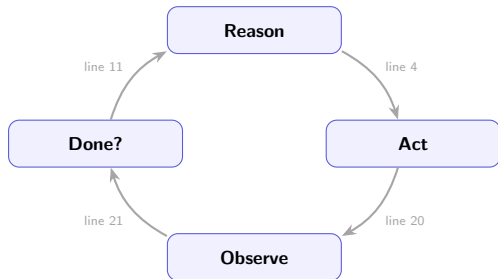
You write this loop once. The model never writes or runs it — it only supplies the replies the loop reads.

```
1 messages = [{"role": "user", "content": "Solve the model and report r."}]
2
3 while True:
4     resp = client.messages.create(
5         model="claude-opus-5",
6         max_tokens=16000,
7         system="You are a computational macroeconomist.",
8         tools=TOOLS,                # schemas, not functions
9         messages=messages,          # the whole list, every time
10    )
11    if resp.stop_reason != "tool_use":    # model is done talking
12        break
13    messages.append({"role": "assistant", "content": resp.content})
14
15    results = []
16    for b in resp.content:                # may be SEVERAL tool calls
17        if b.type == "tool_use":
18            results.append({"type": "tool_result",
19                            "tool_use_id": b.id,        # must match
20                            "content": run_tool(b.name, b.input)})
21    messages.append({"role": "user", "content": results})
```

Not automatic: this file — a person wrote it. **Automatic:** *which* tool, *which* arguments, when to stop — chosen by the model inside `create()`, line 4. **Line 9 with line 21:** the list only grows and is re-sent whole every pass.

ReAct, Named — and Where the Textbook Diagram Is Now Dated

The loop you just read has a name in the literature: **ReAct** — reason, act, observe, repeat (Yao et al., 2022).

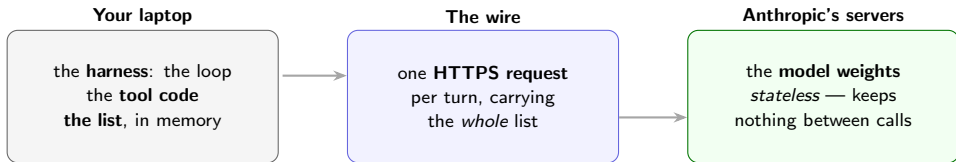


- **Reason** = the model's turn (line 4)
- **Act** = your harness runs the tool (line 20)
- **Observe** = the `tool_result` goes back (line 21)
- **Done?** = the `stop_reason` test (line 11)

Where the 2022 picture is dated. The original ReAct prompted the model to emit literal `Thought: / Action:` text that a parser scraped. Current models emit *structured* `tool_use` blocks and interleave reasoning with tool calls rather than strictly alternating. **Treat ReAct as the ancestor of the loop, not as its wire format** — if you implement the 2022 string-parsing version today you are rebuilding something the API already gives you typed.

So Where Does the Agent Live?

The JSON frame said: “the agent reads it and decides the next action.” That is convenient shorthand. Taken literally it is misleading — no single place contains the agent.



The agent is the loop, not a location. “The agent” names a *behaviour* that appears when a stateless model is driven by a stateful loop. Delete the loop and there is no agent — only a model that answers one question. Read T2’s sentence as: *the harness appended the tool result to the list, and the next request carried that list to the model.*

Why this is the practical frame: it tells you where each cost and limit lives — tokens and the context window on the *wire and the server*, tool permissions and your data on *your laptop* (T2’s privacy boundary), and the memory in a list *you* own and can clear.

Worked Example — GDP Growth by Party

One real research question, traced message by message

The Task — and the Trap Inside It

One prompt, typed once:

“Get US real GDP growth since 2000. Get which party held the White House over the same period. Then plot and rationalize how party affiliation affects GDP growth.”

Three jobs, not one

1. fetch a macro series (*a fact*)
2. build a party timeline (*a fact*)
3. plot it (*mechanical*)
4. explain the relationship (*a causal claim*)

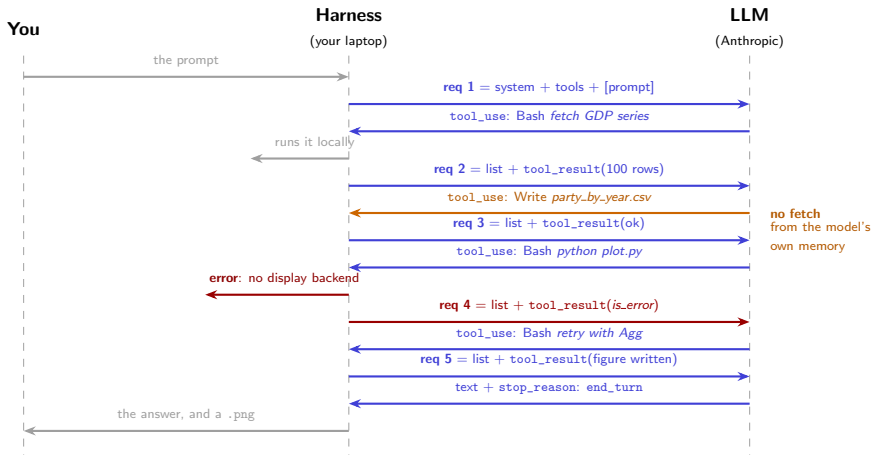
Steps 1–3 an agent can do. Step 4 is where the research is — and where the prompt is booby-trapped.

“Rationalize” is a leading verb. It presupposes an effect and asks for a justification. A compliant model will supply one. Watch, in the trace that follows, the exact turn where a defensible data task turns into an indefensible causal claim — and notice that nothing in the machinery stops it.

A chat model with no tools fails this instantly and invisibly: it will recite GDP numbers from memory. They will look right. Some will be wrong, and none will be checkable.

One Agent: Every Message That Crosses the Wire

Five requests. Each carries the entire list built so far.



Blue crosses the network and is billed; grey stays on your laptop, free. The list grew to eleven messages — req 5 re-sent all of them.

What Went Into req 1 — and Never Changed

This is T2b's initialization, seen from the wire: those five steps exist to build exactly two arguments — then re-sent byte-identical on every later request.

	Assembled from	In this example
system	T2b's init steps 1–4: global settings, CLAUDE.md, .claude/rules/*.md	"You are Claude Code. You have these tools..." + whatever house rules T5b has you write
tools	the schema of every tool the harness is willing to run	Bash, Read, Write, Grep... names, descriptions, argument types — <i>no code</i>

Yes — the model writes the code and asks your machine to run it. A Bash call is the model generating the *text* of a shell command as an argument. It executes nothing; your harness runs that string on your laptop, with your files and permissions.

The safety boundary is not the model's judgement — it is T2b's permission dial, between the generated string and your shell.

system is the only text still guaranteed to be there on turn 200 — everything else drifts or compacts away. That is the mechanical reason CLAUDE.md earns its keep.

Reading the Trace: Four Things Just Happened

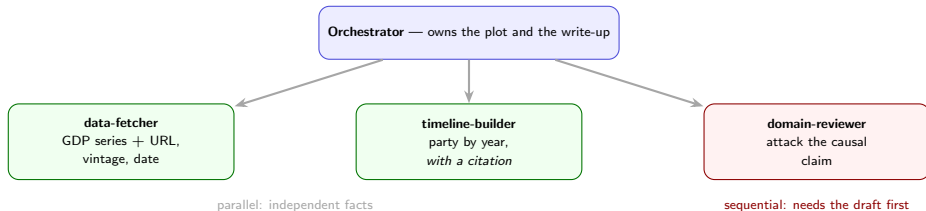
Every claim from the session section shows up in that one picture.

Moment	What it looked like	What it proves
The loop ran itself	five <code>tool_use</code> → <code>tool_result</code> round trips, no human input	nobody scheduled these steps; the model chose each one (above)
The error was recovered	backend error came back as a <code>tool_result</code> with <code>is_error</code> , and the next turn retried	this is “Observe” in ReAct, and why a failed tool must still <i>return</i> (above)
Cost grew quadratically	req 5 re-sent everything from req 1	the list is the cost driver, not the number of questions (above)
One step had no tool	the party timeline was <i>written from memory</i> (orange, previous slide)	the weakest link: in the transcript it looks identical to a fetched fact

Where a referee would stop you. The GDP series has provenance — a URL, a vintage, a retrieval date. The party timeline has none; it is model recall typed into a CSV. **Ask of every number an agent hands you: which tool produced this, and can I re-run it?**

The Same Task with Three Agents

The useful question is not “is this faster” but which steps are independent, and which one needs an adversary.



What fan-out actually buys

- the two *facts* are independent → genuinely parallel
- 100 rows land in data-fetcher's context, **not** the orchestrator's
- one job each, so “cite your source” is enforceable

What it does not buy

- the plot is one small step — splitting it adds turns, not speed
- three agents cost more tokens for the *same* numbers
- nothing here makes the causal claim more true

Only domain-reviewer **changes the answer** — and it cannot run in parallel: it needs something to attack.

What the Reviewer Says — and What No Agent Can Fix

Given the draft, a competent domain-reviewer returns roughly this:

“The correlation is real but you cannot read it as an effect. Since 2000 you have about **six administrations** — your effective n is single digits, not the 100 quarters you plotted. Presidents inherit economies, business cycles are not assigned at random, and Congress is unmodelled. **Blinder and Watson (2016)** find a large gap over 1949–2012 (about **1.79pp** more annual real GDP growth under Democratic presidents, 16 terms) and attribute most of it to *oil shocks, productivity shocks, and foreign demand* — not to policy. Report the correlation, state the identification problem, and drop the word ‘affects’.” **[solid]**

This is the case for multi-agent, stated honestly. Not speed. A second agent whose job is to *disagree* caught a claim the first agent was happy to make — because you told it to.

And the limit. Every agent here inherited your word “*rationalize*”. A more agreeable reviewer would have obliged. **No number of agents fixes a leading prompt** — that is T5c’s Context Dilemma, in one word of your own typing.

The fix is upstream and costs nothing: ask “*is there a relationship, and what would identify it?*” — then the same machinery works for you instead of against you.

Bridge: Put the Loop on Your Own Machine

You have watched the loop on the wire, once with one agent and once with three.

- A session is a list the harness re-sends; the agent is the loop, not a place
- Every number an agent hands you should name the tool that produced it
- **Next (T2b):** put that loop on your own machine — install it, learn its files and commands, run your first session